

Operations

Running the stack, configuring it, and what to check when something is wrong.

The stack

Defined in `docker-compose.yml`.

Service	Image	Port	Depends on
db	postgres:16-alpine	5433 → 5432	—
api	built from <code>api/Dockerfile</code>	4000	db healthy
web	built from <code>frontend/Dockerfile</code>	3000	api healthy

The `web` service is currently commented out, so `docker compose up -d` starts `db` and `api` only. That is deliberate while the client is being worked on: `npm run dev:web` on the host is a hot reload rather than a two-minute image rebuild for every change. Uncomment the block to run the full stack from images — nothing else needs changing, because the API does not depend on it.

Each service waits on the one below being **healthy**, not merely started: `db` answers `pg_isready`, and `api` answers `GET /api/health`. The web container therefore never starts against an API still applying migrations.

Postgres is published on **5433** so it cannot collide with another Postgres on the host. Inside the Compose network it is reached at `db:5432`.

```
npm run docker:up      # build and start db + api (web is commented out)
npm run docker:logs    # follow
npm run docker:down    # stop
```

`docker compose down -v` drops **both** volumes — the database and the uploaded files — which is how you get a clean reseed. Note that it takes every uploaded document and signed copy with it; the seed does not recreate them, because they are made through the app.

Environment

Every value has a working default in `docker-compose.yml`, so the stack runs with no `.env` at all. Copy `.env.example` to `.env` to change any of it.

Variable	Default	Notes
POSTGRES_USER	peoplepay	
POSTGRES_PASSWORD	peoplepay_dev_pw	Change it anywhere real.

Variable	Default	Notes
POSTGRES_DB	peoplepay360	
POSTGRES_PORT	5433	Host port only.
DATABASE_URL	—	Only for running the API on the host. In Compose the API builds its own from the parts above.
API_PORT	4000	
JWT_SECRET	a placeholder	Change it. Every token is signed with it, so rotating it signs everyone out. The API refuses to boot in production if it is under 32 characters.
JWT_EXPIRES_IN	7d	
CORS_ORIGIN	http://localhost:3000	Where the browser loads the web app from.
WEB_PORT	3000	
NEXT_PUBLIC_API_URL	http://localhost:4000	Build-time. Next inlines <code>NEXT_PUBLIC_*</code> , so changing it needs a rebuild, not a restart.
API_URL	http://api:4000	Server-to-server, over the Compose network. Set in the compose file, not <code>.env</code> .
ACS_EMAIL_CONNECTION_STRING	empty	Azure Communication Services. Setting it switches payslip delivery from the outbox to real sending.
ACS_SENDER_ADDRESS	empty	Must be a verified sender on the ACS domain. Azure rejects any other username outright.
SMTP_*, MAIL_FROM	empty	The older fallback, used only when there is no ACS connection string.
STORAGE_ROOT	/data/storage in Compose, ./storage on the host	Where uploaded documents, signed copies, avatars and the logo are written. Backed by the <code>peoplepay_files</code> volume.
AI_BRIDGE_URL	empty	Empty disables the AI features cleanly rather than failing on a refused connection.
AI_BRIDGE_TOKEN	empty	Set it if the bridge is running. Anything that can reach the port can spend money on your Claude account.
AI_TIMEOUT_MS	150000	The bridge keeps its own, shorter timeout under this one.
AVATAR_API_URL	DiceBear PNG endpoint	Where a starter profile picture is fetched from when an employee is created. Empty disables it.
AVATAR_SIZE	256	Pixels.

AI features

Writing a document and reading an upload go through `ai-bridge/`, a small host-side process that runs the Claude CLI and answers over HTTP. The CLI is signed in as a person rather than holding an API key, so the container cannot invoke it and reaches back out to the host instead.

```
cd ai-bridge && AI_CLI_PATH=... AI_BRIDGE_TOKEN=... node server.mjs
```

Set `AI_BRIDGE_URL` and `AI_BRIDGE_TOKEN` in the root `.env`. Leave the URL empty and both features report that AI is not set up, rather than failing on a refused connection. `ai-bridge/README.md` has the rest.

Email delivery

The provider is chosen by what is configured rather than by a flag:

Configured	What happens
<code>ACS_EMAIL_CONNECTION_STRING</code>	Sent through Azure Communication Services.
<code>SMTP_HOST</code> (and no ACS)	Sent through SMTP. Needs <code>npm i nodemailer -w @peoplepay360/api</code> .
Neither	Nothing leaves. The attempt is recorded in the outbox as <code>QUEUED</code> , with the reason on the row, so the bulk-send flow is demonstrable without credentials.

`QUEUED` exists so this last case cannot pass for success. An outbox-only install used to write `SENT` and report every invite as delivered, which meant the one person who could fix the missing credentials had no way to learn they were missing. Nothing but a real delivery is counted as one: `Payslip delivery` reports queued separately from sent, an invite that did not go out leaves `invitedAt` null, and creating an employee hands the one-time password back to whoever created them so the account is still reachable by hand.

Compose reads the `.env` beside `docker-compose.yml`, not `api/.env`. A mail configuration in the wrong one silently resolves to empty, which is exactly the state that produces `QUEUED`.

Either way the payslip PDF is attached, and it is generated **before** sending, so a payslip that cannot be rendered fails rather than arriving as an email with nothing on it.

ACS `beginSend` returns a poller, and the API waits for the operation to finish rather than for the request to be accepted. That is what turns a rejected sender or a bad recipient into a `FAILED` row in the outbox with Azure's own message, instead of a silent non-delivery recorded as sent.

The sender address is the usual thing to get wrong. Azure only accepts usernames configured on the connected domain — `donotreply@yourdomain` typically exists, `people@yourdomain` typically does not, and the error names the username it rejected.

The two API addresses

The web app talks to the API from two places, and they are not the same address:

- `API_URL` — used by the Next.js server when it renders. Inside Compose that is `http://api:4000`, the service name on the internal network.
- `NEXT_PUBLIC_API_URL` — baked into the browser bundle. It must be an address the *user's browser* can reach, so `http://localhost:4000` locally and a public hostname in a deployment.

Getting these the same way round is the usual cause of "works in dev, blank in Docker".

Local development

```
npm install
npm run setup      # start Postgres, build shared, migrate, seed
npm run dev        # API :4000, web :3000
```

`npm run setup` only needs running once, or after a schema change.

The three workspaces are `@peoplepay360/api`, `@peoplepay360/web` (in `frontend/`) and `@peoplepay360/shared`. The shared package resolves by symlink, so editing a type in it is immediately visible to both — but it is **compiled**, so run `npm run build -w @peoplepay360/shared` (or `npm run dev` in that package for watch mode) after changing it.

Database

```
npm run db:migrate  # create and apply a migration
npm run db:seed     # seed
npm run db:reset    # drop, re-migrate, reseed
npm run db:studio   # browse
```

Migrations live in `api/prisma/migrations/` and are applied automatically on container start (`prisma migrate deploy`), followed by a seed if the database is still empty. A fresh volume therefore comes up usable.

The root `.gitignore` once carried a bare `prisma/` rule. That matches at **any depth**, so it silently excluded `api/prisma` — schema, migrations and seed were untracked. It is anchored to `/prisma/` now. Check `git status` after touching ignore rules.

Images

Both Dockerfiles take the **repository root** as their build context, because the shared workspace has to resolve the same way it does locally:

```
docker build -f frontend/Dockerfile .
docker build -f api/Dockerfile .
```

The web image is three stages — install, build, run. The runtime stage carries only Next's `standalone` output plus static assets, so it has no build toolchain and no full `node_modules` tree, and it runs as the `node` user rather than root.

`next.config.ts` sets `outputFileTracingRoot` to the repository root. Without it, tracing starts at `frontend/` and leaves the shared package out of the bundle.

Deployment notes

Nothing here is production-hardened; this is what to change first.

1. `JWT_SECRET` — a long random string, stored as a secret rather than in `.env`.
2. `POSTGRES_PASSWORD` — likewise, and use a managed database rather than the compose volume.
3. `CORS_ORIGIN` and `NEXT_PUBLIC_API_URL` — real hostnames. The second is build-time.
4. **TLS** — terminate in front of both services. The session cookie sets `secure` when `NODE_ENV=production`, so it will not be sent over plain HTTP.
5. **Backups** — there are **two** durable volumes, and both matter. `peoplepay_pgdata` holds the database; `peoplepay_files` holds every uploaded document, signed copy, avatar and the company logo. Backing up only the first leaves rows pointing at files that no longer exist. They are separate volumes precisely so either can be moved or restored without the other.
6. **Object storage** — local disk means one API instance. `StorageService` is the only class that would have to change to put the bytes in S3 or Azure Blob.

Troubleshooting

The web app renders but every list is empty. The browser is reaching the app, but the server cannot reach the API. Check `API_URL`. In Compose it must be `http://api:4000`, not `localhost`.

Sign-in fails with "Cannot reach the API". `apiFetch` turns a connection failure into that message. `docker compose ps` — is `api` healthy? `docker compose logs api` will usually show a database connection error underneath.

A by-id route returns 400. Primary keys are UUIDv7, but several id-bearing columns carry no foreign key and are plain text, so `IsUUID` / `ParseUUIDPipe` will reject valid values. Use `IsEntityId` and `ParseEntityIdPipe` from `api/src/common/validation/entity-id.ts`.

A create fails with "property ... should not exist". The API rejects unknown body fields rather than stripping them. Something is sending a value the DTO does not declare — often a derived one the API computes for itself, like `hoursPerWeek`.

A delete "succeeds" but the row is still there. It was archived, not deleted, because payroll history refers to it. The response carries `{ deleted, archived }`; read it rather than assuming.

Port 5432 already in use. It should not be — the database is published on 5433. If you changed `POSTGRES_PORT`, change `DATABASE_URL` to match.

Changing `NEXT_PUBLIC_API_URL` had no effect. It is inlined at build time. Rebuild the image: `npm run docker:up` rebuilds.

Type errors after editing `shared`. It is compiled. Rebuild it: `npm run build -w @peoplepay360/shared`.